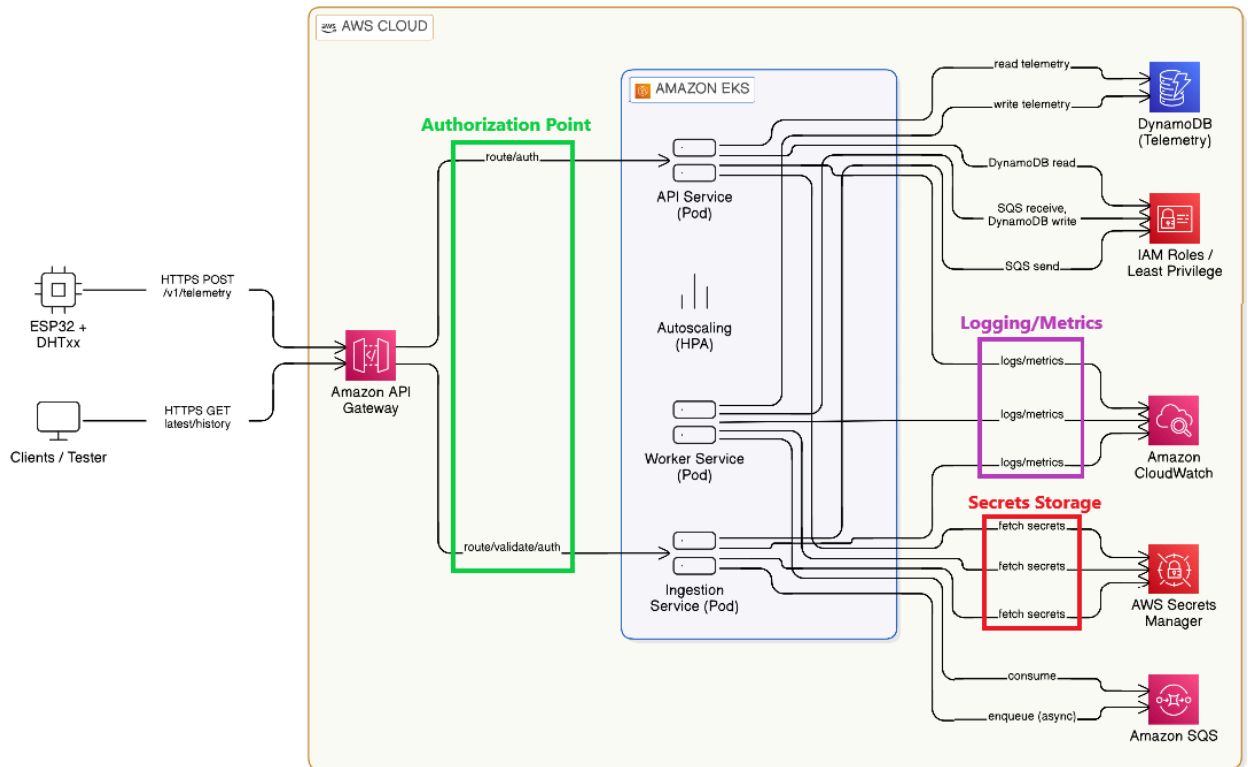


Project Critical Design Review
Cloud-Based IoT Environmental Telemetry Platform
Seid Cubro - Charles Shoppel

1. Updated Architecture Diagram



2. Requirements Traceability Table

Requirement	Description	Component(s)	Milestone	Verification/ Evidence
FR-1	Ingest telemetry data	API Gateway, Ingestion Service	M4	POST /v1/telemetry returns 202
FR-3	Retrieve latest telemetry	API Service	M4	GET /v1/devices/{id}/telemetry/latest returns 200
FR-6	Asynchronous processing	SQS, Worker Service	M7	Queue messages processed by worker
FR-8	Cloud deployment	EKS, Ingress	M6	Services reachable via Kubernetes

				ingress
FR-9	Scalability	Kubernetes HPA	M6	HPA scales pods under load
NFR-3	Secure access	API Gateway, IAM, Secrets Manager	M5	Auth enforced, secrets stored
NFR-4	Observability	CloudWatch, Health Endpoints	M5	Service logs and health endpoint validated locally
NFR-7	Cost controls	AWS Budgets, cleanup policies	M8	Budget alerts, teardown verified

3. Security Baseline

Authentication & Authorization:

- Primary authentication point: Amazon API Gateway
- Auth mechanism: API key-based authentication for device ingestion
- Rationale:
 - Lightweight and compatible with ESP32 devices
 - Native support in API Gateway
 - Sufficient for project threat model and scope

API Gateway validates incoming requests before routing traffic to backend services, ensuring only authorized devices and clients can access the system.

Secrets Management:

- Secrets store: AWS Secrets Manager
- Managed secrets include:
 - API Keys
 - Service configuration values
- Access model:
 - Kubernetes workloads access secrets via IAM roles
 - No secrets are hardcoded in containers or source code

Rotation Strategy:

Secrets can be rotated by updating Secrets Manager and API Gateway without redeploying application services.

Network Security and Trust Boundaries:

- Primary trust boundary: API Gateway
 - This is where public internet traffic enters the system
- Internal services:
 - EKS workloads run in private subnets
 - DynamoDB is accessed using IAM roles, not network credentials
- Transport security:
 - TLS terminates at API Gateway
 - Internal service communication occurs over secure channels

This design minimizes the public attack surface and enforces least-privilege access across all components.

Security Readiness Summary:

- Authentication enforced at system entry point
- Secrets centrally managed and protected
- IAM-based access controls used throughout
- No plaintext credentials stored in code containers

This baseline establishes a secure foundation for cloud deployment in later milestones.

4. Observability Baseline

Logging

- Logging framework: Application-level logging in services
- Log content includes:
 - Incoming requests
 - Processing events
 - Errors and failures

Logs are visible during local prototype execution and are designed to integrate with Amazon CloudWatch Logs during cloud deployment.

Key Metrics for System Monitoring

- API request rate and error rate (5xx)
- Request latency (p95)
- SQS queue depth
- Worker processing error rate
- Pod CPU utilization

These metrics provide visibility into system health, performance and scaling behavior.

Health Checks

- Each service exposes a /health endpoint
- Health endpoints are used to:
 - Verify service availability
 - Support Kubernetes readiness and liveness checks

Health endpoints were validated during the local prototype in M4.

Sample Alert Thresholds (Planned)

- Queue depth > 100 messages
- API 5xx error rate > 2%
- P95 latency > 1 second
- Worker error rate > 1%

Alerts will be implemented using CloudWatch Alarms in later milestones.

Observability Readiness Summary

- Logs available and validated locally
- Health endpoints implemented and verified
- Metrics identified with clear operational intent
- Alert thresholds defined

This baseline ensures the system is observable and diagnosable prior to full cloud deployment.